1. ¿Qué evidencia del negocio influyó más en la selección del patrón?

Influyó sobre todo que la orden de servicio tiene **ocho estados con transiciones restringidas, nueve excepciones y un historial obligatorio (N2)**, junto con reglas que cruzan varios conceptos: la orden solo nace de una cotización aceptada (US-06), la unidad debe tener capacidad suficiente y el operador estar disponible (US-07, G2), y la entrega exige receptor y evidencia (US-12 y US-13). Con Transaction Script esas validaciones se repetirían en cada operación; con Domain Model quedan en la raíz del agregado. Además, el BRD deja pendientes que cambiarán, lo que favorece reglas aisladas y reemplazables.

2. ¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?

La más difícil fue **dónde vive la regla de capacidad de la unidad**, porque el BRD exige capacidad suficiente pero no define límites y prohíbe inventarlos. No la pusimos dentro de Unidad (habría que editar la clase con cada política) ni en el handler (mezclaría negocio con coordinación). La decidimos como interfaz PoliticaDeCapacidad que usa el servicio de dominio AsignadorDeRecursos, y los límites propios quedan como el supuesto S-01 (ADR-004). También fue dudoso el límite del agregado, y decidimos que orden y seguimiento del servicio sean un solo agregado OrdenDeServicio (ADR-005).

3. ¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?

Una regla central es que **la entrega solo se registra si el servicio llegó a destino y existen receptor y evidencia**; vive en OrdenDeServicio.registrar_entrega. Si estuviera en el handler o en la interfaz, otro caso de uso o una consulta podría saltarse la validación, la regla se duplicaría y la orden podría quedar ENTREGADO sin evidencia, incumpliendo N4. Igual con la regla de que la orden solo se genera tras la aceptación de la cotización.

4. ¿Qué cambió después de la revisión con SOLID?

Se reemplazó una condición fija de capacidad por la abstracción PoliticaDeCapacidad (principio abierto/cerrado, ADR-004). Se confirmó que dominio y aplicación no importan infraestructura (inversión de dependencias) y que lectura y escritura están en interfaces separadas. **Tres puntos no se corrigieron y los dejamos como riesgos aceptados**: el tamaño de OrdenDeServicio, que IUnitOfWork expone los cuatro repositorios, y que cada interfaz tiene una sola implementación.

5. ¿Qué propuesta de la IA rechazaron o corrigieron y por qué?

Corregimos varias: (1) el ADR de persistencia asumía clases de base de datos («Bd…») que el BRD no define; las cambiamos por interfaces del dominio e implementaciones en memoria. (2) El BPMN usaba IDs distintos a la matriz de la U2; los unificamos. (3) En la secuencia 2 la rama de error seguía guardando; ahora termina en revertir y no publica eventos. (4) Nombres que no coincidían con el glosario; actualizamos el glosario. Y para los pendientes del BRD, elegimos supuestos explícitos en lugar de inventar reglas.

6. ¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?

Domain Model exige **más diseño previo, más clases y una curva de aprendizaje**, y hay riesgo de sobre-modelar; lo mitigamos modelando solo lo que aparece en el BRD. Aceptamos también que OrdenDeServicio sea una clase grande, que haya abstracciones con una sola implementación hoy (PoliticaDeCapacidad), y que parte del comportamiento dependa de supuestos que todavía no valida el stakeholder.